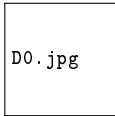


Lecture 33: Advanced SQL

Introduction to Window Functions & Ranking Mechanics

Z2004: Database Management Systems



D0.jpg

IIT Madras Zanzibar

Thursday, 11 June 2026

Analyzing Dynamic Multi-Row Relationships Privately

- ➊ Pop Quiz on CTEs & Recursion.
- ➋ What exactly is a Window Function?
- ➌ GROUP BY vs. Window Functions.
- ➍ Mastering the OVER() clause.
- ➎ Practical performance guidelines.
- ➏ Deep dive into ROW_NUMBER(), RANK(), DENSE_RANK(), and NTILE().

CTEs & Recursion Recap Quiz

Write the exact SQL queries for the SchoolDB.

Tables: *Student*(*SID*, *Name*, *Dept*, *GPA*), *Employee*(*ID*, *Name*, *ManagerID*).

- 1 **Standard CTE:** Write a query using a CTE named `HighAchievers` that selects all students with a GPA over 3.8, then query everything from that CTE.
- 2 **CTE Join:** Define a CTE that calculates the average school GPA, then write an outer query that selects all students whose personal GPA is below that average.
- 3 **Multiple CTEs:** What keyword separates multiple CTE declarations within a single `WITH` clause statement?
- 4 **Recursive CTE Base:** In a recursive CTE designed to traverse a tree hierarchy, what is the technical name given to the initial, non-recursive query block?
- 5 **Recursive Join:** Write the recursive component query of a CTE named `Org` that links an `Employee` table back to the `Org` CTE using a manager relationship.

 You have 7 Minutes. No notes, no talking, write on paper!

Solutions!

1. Standard CTE

```
WITH HighAchievers AS (  
    SELECT * FROM Student WHERE GPA > 3.8  
)  
SELECT * FROM HighAchievers;
```

2. CTE Join

```
WITH SchoolAvg AS (  
    SELECT AVG(GPA) AS AvgGPA FROM Student  
)  
SELECT s.Name, s.GPA  
FROM Student s  
CROSS JOIN SchoolAvg a  
WHERE s.GPA < a.AvgGPA;
```

3. Separator Keyword

- **Answer:** A basic comma (,). You only declare the WITH keyword once at the very beginning!

4. Technical Term

- **Answer:** The Anchor Member.

5. Recursive Join Member

```
-- This statement goes below the UNION ALL  
SELECT e.ID, e.Name, e.ManagerID  
FROM Employee e  
INNER JOIN Org o ON e.ManagerID = o.ID
```

This is the part that “expands” the hierarchy by pulling employees (e) whose ManagerID matches an already-found person in the CTE (o). Typically it sits under a UNION ALL after the anchor query.

What is a Window Function?

Imagine you are looking at your dataset printed on a long sheet of paper.

The Old Way (GROUP BY):

- ▶ You take a pair of scissors, cut the rows by department, toss them into individual buckets, and blend them into a single value.
- ▶ **Result:** Individual names and identities are permanently destroyed.

The Window Way:

- ▶ You slide a physical piece of clear glass (a window frame) over a specific subset of rows.
- ▶ You look through the glass, compute an aggregate value (like an average), and write that result right next to the original row.
- ▶ **Result:** No rows are destroyed; individual data details remain intact!

💡 **Core Definition:** A Window Function performs a calculation across a set of table rows that are somehow related to the current row, without losing row identities.

GROUP BY vs. Window

Let's look at the functional difference between these two processing techniques.

Using standard GROUP BY:

```
SELECT Dept , AVG(GPA)
FROM Student
GROUP BY Dept ;
```

Output format:

Dept	AVG(GPA)
CSE	3.50
EEE	3.10

Rows collapse entirely. We cannot display individual student names here!

Using a Window Function:

```
SELECT Name , Dept , GPA ,
       AVG(GPA) OVER() AS SchoolAvg
FROM Student ;
```

Output format:

Name	Dept	GPA	SchoolAvg
Asha Ali	CSE	3.80	3.44
Omar Said	CSE	3.20	3.44
Zahra Musa	EEE	3.32	3.44

Every individual row is kept, and the total school average is seamlessly appended to every line!

Window Function Example: Dept Average & Rank (No Rows Lost)

Let's compute a **department average** and a **rank within each department** *without collapsing rows*.

Using a Window Function:

```
SELECT Name, Dept, GPA,
       AVG(GPA) OVER (PARTITION BY Dept) AS DeptAvg,
       RANK() OVER (PARTITION BY Dept
                   ORDER BY GPA DESC) AS DeptRank
FROM Student;
```

Output format:

Name	Dept	GPA	DeptAvg	DeptRank
Ana	CSE	3.90	3.55	1
Ben	CSE	3.20	3.55	2
Cam	EEE	3.70	3.40	1
Dia	EEE	3.10	3.40	2

Every individual row is kept, and analytic results (DeptAvg, DeptRank) are appended next to it.

Anatomy of a Window Function: The OVER() Clause

How does the database engine distinguish between a traditional aggregate function and a window function? The presence of the OVER() keyword.

```
SELECT Name, AVG(GPA) OVER ( ) FROM Student;
```

- ▶ **The Function Component (AVG(GPA))**: Specifies the mathematical operation to execute through our window frame (e.g., SUM, COUNT, AVG, or specialized ranking tools).
- ▶ **The Window Operator (OVER)**: Directs the database query engine to construct an analytical window partition space.
- ▶ **The Configuration Body (())**: Instructs the system on how to partition or sort the data frame before evaluating the mathematical calculation. Leaving it empty () treats the entire table as one single window frame!

When to Use (and When NOT to Use)

To build reliable software architectures, you must understand exactly when to deploy window logic and when it will hurt database engine performance.

✔ Ideal Use Cases:

- ▶ **Running Totals / Moving Averages:** Calculating real-time financial tracking logs.
- ▶ **Deduplication:** Identifying and purging precise duplicate records.
- ▶ **Comparative Baselines:** Comparing individual items instantly against total group statistics.
- ▶ **Leaderboards:** Ranking entities across complex multi-variable conditions.

✘ When to Avoid / Constraints:

- ▶ **The WHERE Clause Block:** Window functions evaluate *after* filtering occurs. You **cannot** place them inside a standard **WHERE** clause directly!
- ▶ **Heavy Write Environments:** Creating multi-layer window evaluations over massive datasets requires standard memory sorting operations, which can slow down real-time transaction processing.

Ranking functions: ROW_NUMBER vs. RANK vs. DENSE_RANK

- ▶ ROW_NUMBER(): assigns a unique sequence number (1,2,3,...) to each row in the ordered window *even if values tie*.
- ▶ RANK(): ties get the *same* rank, and the next rank *skips* (creates gaps).
- ▶ DENSE_RANK(): ties get the *same* rank, but the next rank *does not skip* (no gaps).

Quick tie example (sorted by GPA DESC):

Name	GPA	ROW_NUMBER	RANK	DENSE_RANK
Noah	3.92	1	1	1
Asha	3.80	2	2	2
John	3.80	3	2	2
Lina	3.74	4	4	3

The Ranking Trio: Differences Exposed

Let's apply these concepts by ranking students based on their GPAs using `ROW_NUMBER()`, `RANK()`, and `DENSE_RANK()`. Note how they uniquely handle ties!

```
SELECT Name, GPA,
       ROW_NUMBER() OVER (ORDER BY GPA DESC) AS RowNum,
       RANK() OVER (ORDER BY GPA DESC) AS Rnk,
       DENSE_RANK() OVER (ORDER BY GPA DESC) AS DenseRnk
FROM Student;
```

Visual Comparison Output Matrix:

Name	GPA	ROW_NUMBER()	RANK()	DENSE_RANK()	Behavior Explanation
Noah Kim	3.92	1	1	1	Absolute top rank position.
Asha Ali	3.80	2	2	2	Duplicate item values tie for position.
John Doe	3.80	3	2	2	Duplicate item values tie for position.
Lina Zhang	3.74	4	4	3	RANK skips spaces; DENSE stays tight!

Why statistical distributions? (Quantiles)

Statistical distribution = how values are spread across the population (center, spread, skew, tails).

Why we care (in data systems):

- ▶ **Summarize** a large dataset quickly ("what does typical look like?").
- ▶ **Compare** entities to peers ("top 25%" / "bottom 10%").
- ▶ **Bucket** users/items for reporting, grading curves, segmentation, A/B analysis.

Quantiles (cut points on sorted data):

- ▶ **Quantile**: split ordered data into equal-sized groups.
- ▶ **Quartiles**: 4 groups (each $\approx 25\%$).
- ▶ **Quintiles**: 5 groups (each $\approx 20\%$).
- ▶ **Percentiles**: 100 groups (each $\approx 1\%$); e.g., 90th percentile = at/above 90% of values.

Statistical Distribution using NTILE()

Dividing records into ordered mathematical buckets

The `NTILE(n)` function divides your sorted dataset as evenly as possible into n separate buckets (e.g., quartiles, quintiles, percentiles).

```
-- Separate our school body straight into 4 clean performance quartiles
SELECT Name, GPA,
       NTILE(4) OVER (ORDER BY GPA DESC) AS Quartile
FROM Student;
```

Quartile Output Layout Visualization:

Name	GPA	Quartile	Bucket Category Allocation
Noah Kim	3.92	1	Bucket 1 (Top 25% Tier)
Asha Ali	3.80	1	Bucket 1 (Top 25% Tier)
Lina Zhang	3.74	2	Bucket 2 (Upper Middle Tier)
Arjun Singh	3.66	2	Bucket 2 (Upper Middle Tier)

Key takeaways (L33: Window Functions I)

- ▶ **Preserving Row Identity:** Window functions execute complex aggregate transformations while keeping individual row details intact.
- ▶ **The OVER() Operator:** Triggers window processing logic. Leaving it blank OVER() evaluates the calculation across the entire table.
- ▶ **ROW_NUMBER():** Sequentially numbers rows without taking ties into account.
- ▶ **RANK():** Awards identical numbers to ties, skipping positions afterward to account for the duplicates.
- ▶ **DENSE_RANK():** Awards identical numbers to ties, maintaining a continuous sequence with no gaps.
- ▶ **NTILE(n):** Groups sorted tuples into clean, equal-sized sequential buckets.

Tomorrow: Moving Windows, Stored Procedures, and Triggers!